

STAT:5400 Section 2.3 — Functions, Loops, and Plots

Boxiang Wang

2026-09-09

1 Functions

1.1 Write your own functions

- Structure of a function.

1.1.1 R

```
function (<arguments>) {  
  <body of function>  
  <object to return>  
}
```

- Three parts of the function: **formals**, **body**, **environment**.

```
myfunc <- function(x) sin(x)  
myfunc
```

```
formals(myfunc)  
body(myfunc)  
environment(myfunc)
```

1.1.2 Python

```
def <name>(<arguments>):
    <body of function>
    return <object to return>
```

- The same three parts, under different names: **signature**, **code**, **globals**.

```
import math
def myfunc(x):
    return math.sin(x)
myfunc
```

```
import inspect
print(inspect.signature(myfunc))      # formals
print(myfunc.__code__)                # body, already compiled
print(myfunc.__globals__ is globals()) # environment
```

Typing the name of an R function prints its body, because R keeps the source text with the function. Python keeps only compiled bytecode, so the same line prints an address.

1.2 Quadratic equation solver (a bad example)

A question from STAT 5400 proficiency exam:

1.2.1 R

```
quadratic <- function(a, b, c){
  discrim <- b * b - 4 * a * c
  mult <- c(1, -1)
  solve <- (-b + mult * sqrt( discrim ) ) / (2 * a)
  return(solve)
}
```

```
# usage
quadratic(2, 4, 1)
```

1.2.2 Python

```
import math
def quadratic(a, b, c):
    discrim = b * b - 4 * a * c
    solve = [(-b + m * math.sqrt(discrim)) / (2 * a) for m in (1, -1)]
    return solve
```

```
# usage
quadratic(2, 4, 1)
```

On these coefficients it gives the right answer. It is still a bad function: the name `solve` hides an R function, the argument `c` hides `c()`, and nothing is checked.

What would it return when `a` is zero, or when the discriminant is negative?

1.3 Quadratic equation solver (a better example)

1.3.1 R

```
QuadraticSolver <- function(aval, bval, cval){
  discrim <- bval * bval - 4 * aval * cval
  if (discrim < 0){
    sol <- c(NA, NA)
    errflag <- 1
  } else {
    if (aval != 0) {
      mult <- c(1, -1)
      sol <- (-bval + mult * sqrt(discrim)) / (2 * aval)
      errflag <- 0
    } else {
      sol <- c(NA, NA)
      errflag <- 2
    }
  }
  return(list(solution=sol, errflag=errflag))
}
```

```
QuadraticSolver(2, 4, 1)
```

1.3.2 Python

```
import math
def QuadraticSolver(a, b, c):
    discrim = b * b - 4 * a * c
    if discrim < 0:
        sol = [None, None]
        errflag = 1
    else:
        if a != 0:
            root1 = (-b + math.sqrt(discrim)) / (2 * a)
            root2 = (-b - math.sqrt(discrim)) / (2 * a)
            sol = [root1, root2]
            errflag = 0
        else:
            sol = [None, None]
            errflag = 2
    return sol, errflag
```

```
QuadraticSolver(2, 4, 1)
```

i Why the flags matter more in R than in Python

Without the checks, R answers anyway.

Take the checks out and give the function `a = 0`. R divides by zero and returns `NaN` and `-Inf`. Give it a negative discriminant and R returns two `NaN` values with a warning that is easy to miss. Either way you get something number-shaped, and you can carry it into the next computation without noticing.

Python stops at the line that is wrong: `ZeroDivisionError` in the first case, and `ValueError: math domain error` in the second.

So `errflag` is doing more work in R. In Python the interpreter already refuses; in R the only thing standing between a bad input and a bad number is the code you wrote.

1.4 A bug the flags do not catch

Both versions use the formula from the textbook. Try coefficients that are far apart.

💡 Predict — the two roots of a quadratic must multiply to c/a , which is 1 here. Will they? Done when you can say which of the two roots is wrong.

1.4.1 R

```
r <- QuadraticSolver(1, 1e8, 1)$solution
r
prod(r)    # must be c/a = 1
```

1.4.2 Python

```
r = QuadraticSolver(1, 1e8, 1)[0]
print(r)
print(r[0] * r[1])    # must be c/a = 1
```

Solution

The product is 0.745, not 1, and no error was raised.

The large root is fine. The small one should be -1.0×10^{-8} and comes back as -7.45×10^{-9} . The cause is subtraction. With b large, $\sqrt{b^2 - 4ac}$ is almost exactly b , so $-b + \sqrt{b^2 - 4ac}$ takes the difference of two nearly equal numbers. The digits they share cancel, and what is left is mostly rounding error. No flag in the function can see this.

The standard fix computes the safe root first, then gets the other one from the product.

```
q <- -(bval + sign(bval) * sqrt(discrim)) / 2
sol <- c(cval / q, q / aval)
```

That version returns -1.0×10^{-8} , and the product becomes 1.

 Vibe — give `QuadraticSolver` to Copilot and ask it to make the function robust, then test the result on `(1, 1e8, 1)`. Starter files: [vibe/s2p3_quadratic_robust.R](#) and [vibe/s2p3_quadratic_robust.py](#). Expect type checks, a docstring, and a message for the negative discriminant. Expect the unstable formula to survive. Models fix the errors that have names, and this one has no name until you give it one. That division of labor runs through the rest of the course.

1.5 Putting a function in its own file

This program is saved in a file called “quadratic.R” (or “quadratic.py”). Running that file puts the function in memory.

1.5.1 R

```
source("quadratic.R")
QuadraticSolver(2, 4, 1)
```

1.5.2 Python

```
from quadratic import QuadraticSolver
QuadraticSolver(2, 4, 1)
```

R runs the whole file and keeps everything it created. Python imports the file as a module, and you name the pieces you want.

You may also call a function with a list of arguments already collected in one object.

1.5.3 R

```
args <- list(aval=2, bval=4, cval=1)
do.call(QuadraticSolver, args)
```

1.5.4 Python

```
args = {"a": 2, "b": 4, "c": 1}
QuadraticSolver(**args)
```

The two stars in front of a Python dictionary mean “spread these out as named arguments”. One star does the same for a list, filling the arguments in order.

1.6 Local and global variables (name masking)

1.6.1 R

```
y <- -1
```

```
myfun1 <- function(x=1, y=1) {
  y <- y + 1; return(y)
}
```

```
myfun2 <- function(x=1) {  
  y <- y + 1; return(y)  
}
```

```
myfun3 <- function(x=1) {  
  y <<- y + 1; return(y)  
}
```

```
myfun1(1); print(y)
```

```
myfun2(1); print(y)
```

```
myfun3(1); print(y)
```

1.6.2 Python

```
y = -1
```

```
def myfun1(x=1, y=1):  
    y += 1  
    return y
```

```
def myfun2(x=1):  
    y = y + 1  
    return y
```

```
def myfun3(x=1):  
    global y  
    y += 1  
    return y
```

```
print(myfun1(1)); print(y)
```

```
print(myfun2(1)); print(y)
```

```
print(myfun3(1)); print(y)
```

💡 Predict — myfun2 is the same three lines in both languages, and y is -1 outside. Say what each version returns before you run it. Done when you can explain why one of them never returns anything at all.

Solution

R returns 0. Python raises `UnboundLocalError`.

R finds no `y` inside the function, so it walks outward to the global `-1`. The result is stored in a new local `y`, and the global one is untouched.

Python decides before the function runs. The body assigns to `y`, so `y` is local everywhere in it, including on the line that creates it. Reading it before it exists is the error.

`myfun3` is where the two agree. `<<-` and `global` both mean the outer name, and both set `y` to 0 for good. Avoid them.

1.7 A confusing example (function vs variable)

Think about what the following code outputs?

1.7.1 R

```
c <- 1
c(c = c)
```

- R is smart; it knows when a function or a variable is needed.
- However, it may cause confusing code and this is something you should avoid.

1.7.2 Python

```
list = [1, 2, 3]
list((4, 5))
```

- Python is not smart here; one name holds one thing at a time.
- The name `list` now holds your data, so the function `list` is gone until you `del list`.

```
del list
list((4, 5))
```

Same lesson, opposite behavior. R separates the two lookups and recovers; Python does not, and shadowing a built-in name is one of the most common bugs in student code.

1.8 Lazy evaluation

- Use `=` rather than `<-` when assign function arguments.

1.8.1 R

```
x <- 0
myfun4 <- function(x) print(x)
myfun4(x<-1)
x
```

```
x <- 0
myfun5 <- function(x) print(x)
myfun5(x=1)
x
```

```
x <- 0
myfun6 <- function(x) print("")
myfun6(x<-1)
x
```

1.8.2 Python

Python has no lazy evaluation: an argument is computed before the call starts, whether the function uses it or not. The walrus operator `:=` assigns inside an expression, which lets us run the same experiment.

```
x = 0
def myfun4(x): print(x)
myfun4(x := 1)
print(x)
```

```
x = 0
def myfun6(x): print("")
myfun6(x := 1)
print(x)
```

The third R block is the point. `myfun6` never uses `x`, so R never evaluates the argument and the global `x` stays 0. Python evaluates it first, so `x` becomes 1 either way.

1.9 Infix and prefix functions

Every operation is really a function call in both languages:

1.9.1 R

```
class(`+`)
```

We may use it as a regular function:

```
`+`(3, 4)
```

Subsetting:

```
letters[2]  
`[(letters, 2)
```

1.9.2 Python

```
import operator  
type(operator.add)
```

```
print(operator.add(3, 4))
```

```
letters = [chr(i) for i in range(ord("a"), ord("z") + 1)]  
print(letters[1])  
print(operator.getitem(letters, 1))
```

R lets you name the operator itself with backticks. Python keeps the plain-function versions in the `operator` module instead. Note the index: R counts from 1, Python from 0.

1.10 Writing your own infix functions

```
`%+%` <- function(c1, c2) paste(c1, c2)
```

```
"Hawkeye" %+% "State"  
"STAT" %+% 5400
```

```
`%**` <- function(c1, i)  
  paste(rep(c1, i), collapse=", ")  
"Olay" **% 3
```

No Python counterpart: you cannot invent a new operator in Python. You can only change what an existing one does for your own class, by defining a method such as `__add__`.

1.11 Use a clean environment

1.11.1 R

```
rm(list=ls())
```

- In fact, `ls()` lists the object in the global environment `.GlobalEnv()`.
- Try `ls.str()` to get a more detailed list.

1.11.2 Python

```
del myfun1, myfun2
```

- `dir()` lists the names in the current namespace; `globals()` gives the names with their values.
- Python has no one-line “delete everything”; restarting the kernel is the normal way.

Do not run these on the page: they would erase the objects the blocks above created.

2 Loop

2.1 Looping

2.1.1 R

```
for (i in 1:10) print(i)
```

```
for (i in seq_along(letters[1:10])) print(i)
```

```
# "for" itself is essentially a function  
`for`(i, 1:10, print(i))
```

```
for (j in c(1, 37, 81)) print( j/2 )
```

```
# while loop: don't know # of iterates in advance
i = 1
while (1) {
  print(i)
  i <- i + 1
  if (i > 10) break
}
```

2.1.2 Python

```
for i in range(1, 11):
  print(i)
```

```
# enumerate is the counterpart of seq_along
for i, ch in enumerate(letters[:10]):
  print(i, ch)
```

```
for j in [1, 37, 81]:
  print(j / 2)
```

```
i = 1
while 1 :
  print(i)
  i += 1
  if i > 10: break
```

The third R block has no Python counterpart: `for` in Python is a statement, not a function you can call by name.

2.2 Comments on the range function in Python

The `range` function in Python manages the list in a memory-efficient way.

2.2.1 R

```
object.size(1:5400)
object.size(seq(5400))
```

2.2.2 Python

```
import sys
print(sys.version_info.major)
print(sys.getsizeof(range(5400)))
```

In Python 2 the same call returned 43272 bytes, because **range** built the whole list. R stores 1:5400 as a compact sequence too, and only writes out all the numbers when you change one of them.

2.3 The apply family of functions in R

- **apply** lets you apply the same function to each row or column of a matrix, and returns results in a vector.
- **lapply** applies same function to each member of a list.
- **sapply** does the same job as **lapply** but returns a vector.
- **tapply** functions on a group level of a variable.

The **apply** family is an R idea with no single Python counterpart. NumPy does the same jobs with an **axis** argument, and lists are handled by comprehensions; both appear in the tabs below.

👉 By hand — a chatbot was asked for the fast way to get the column means of a large matrix. Its answer is in the comments, and two claims in it are wrong. Run the block and let the clock decide. Done when you can name both wrong claims. No AI for this block.

```
# Chatbot: "Use apply(A, 2, mean). apply is vectorised, so it runs at C
# speed and is much faster than writing the loop yourself. colMeans is
# just a convenience wrapper around apply."
A <- matrix(1:1000000, 1000, 1000)
system.time(apply(A, 2, mean))
system.time(colMeans(A))
```

i Solution

colMeans wins. Both claims are wrong.

apply is not vectorised. It is an R loop with the loop written for you, so the per-column work still happens in R. And **colMeans** is not a wrapper around **apply**: it is separate C code that walks the matrix once.

In the browser **colMeans** is usually a few times faster here; on R installed on your own machine the gap is much wider. Read the **elapsed** column only — the browser has no

CPU clock, so `user` and `system` always print as zero.
The lesson is the one behind the whole section. “Vectorised” describes where the loop runs, not whether you can see it.

The `apply` family is not the most efficient way in practice.

2.3.1 R

```
A <- matrix(1:1000000, 1000, 1000)
```

```
system.time(apply(A, 2, mean))
```

```
system.time(for(i in 1:1000) mean(A[,i]))
```

```
system.time(colMeans(A))
```

2.3.2 Python

```
import numpy as np, timeit
setup = "import numpy as np; A = np.arange(1, 1000001).reshape(1000, 1000, order='F')"
```

```
print(timeit.timeit("[A[:, i].mean() for i in range(1000)]",
    setup=setup, number=3))
```

```
print(timeit.timeit("A.mean(axis=0)", setup=setup, number=3))
```

`A.mean(axis=0)` is the NumPy version of `colMeans`, and it wins for the same reason: the loop over columns happens in compiled code, not in the language you are typing.

2.4 `lapply` and `sapply`

2.4.1 R

```
l1 <- list(a1=1L:3L, a2=letters[1:3], a3="abc")
lapply(l1, length)
lapply(l1, typeof)
```

```
unlist(lapply(l1, length))
sapply(l1, length)
```

- Check the source code of `sapply`.
- Apply `lapply` on a list of functions.

```
funcs = list(mean, median, max)
lapply(funcs, function(f) f(1:10))
```

2.4.2 Python

```
l1 = {"a1": [1, 2, 3], "a2": ["a", "b", "c"], "a3": "abc"}
print({k: len(v) for k, v in l1.items()})
print({k: type(v).__name__ for k, v in l1.items()})
```

```
print([len(v) for v in l1.values()])
print(list(map(len, l1.values())))
```

```
import numpy as np
funcs = [np.mean, np.median, max]
print([f(list(range(1, 11))) for f in funcs])
```

A comprehension keeping the keys is `lapply`; one keeping only the values is `sapply`. Functions are ordinary objects in both languages, so a list of them is nothing special.

2.5 Reduce function in R

We may use `Reduce` to write our own `factorial` function.

2.5.1 R

```
n <- 10
factorial(n)
Reduce('*', seq(n))
```

How can we vectorize our `factorial` function?

```
factorial(c(5, 8, 10))
```

2.5.2 Python

```
import functools, operator, math
n = 10
print(math.factorial(n))
print(functools.reduce(operator.mul, range(1, n + 1)))
```

`math.factorial` takes one number at a time. The vectorized version lives in SciPy.

```
from scipy.special import factorial
print(factorial([5, 8, 10]))
```

R's `factorial` is vectorized already, because almost everything in R is. In Python you have to pick a function that was written for arrays.

2.6 How big can a factorial get?

👉 By hand — a chatbot was asked for a factorial that works for large inputs. Its answer is the comment inside the block, and it makes two claims. One of them is true in Python and false in R. The other is bad advice in both. Run the cells and decide which is which. Done when you can say what each language returns for 171. No AI for this block.

2.6.1 R

```
# Chatbot: "Write it recursively, it is cleaner. Big inputs are
# no problem: these languages switch to exact big integers."
factorial(170)
factorial(171)
prod(1:171)
```

```
# how deep R lets calls nest before it stops
getOption("expressions")
```

2.6.2 Python

```
# Chatbot: "Write it recursively, it is cleaner. Big inputs are
# no problem: these languages switch to exact big integers."
import math
print(math.factorial(171))
print(len(str(math.factorial(171))), "digits")
```

```
import sys
print(sys.getrecursionlimit())
def fact_rec(n):
    return 1 if n == 0 else n * fact_rec(n - 1)
try:
    fact_rec(2000)
except RecursionError as e:
    print("RecursionError:", e)
```

Solution

One claim is half true. The other is bad advice everywhere.

Big integers: true in Python, false in R. R stores every number as a double, which stops at about 1.8×10^{308} . So `factorial(170)` is finite, `factorial(171)` is `Inf`, and `prod(1:171)` is `Inf` as well, with no warning. Python integers grow as long as memory allows, so `math.factorial(171)` is exact and has 310 digits.

Recursion: bad advice in both. A recursive factorial calls itself once per step, so `fact_rec(2000)` stacks two thousand calls before the first one can finish. Both languages cap that stack, and both let you read the cap: the two cells above print it. On a desktop install `sys.getrecursionlimit()` is 1000 and this function stops working at a depth of 997, while `getOption("expressions")` is 5000 and R stops at 4986 with “evaluation nested too deeply”. So 2000 is past Python’s cap but inside R’s, which is why Python raises `RecursionError` here and R instead gets deep enough to overflow to `Inf`. A loop or `Reduce` stacks nothing, so it has no depth limit and is faster than either.

When you need factorials of large numbers, work on the log scale: `lfactorial` in R, `math.lgamma` in Python. Neither one overflows.

2.7 Timing the three versions

2.7.1 R

Run in Codespace — `microbenchmark` needs a precise clock, which a browser does not have.

```
MyFactorial1 <- function(nseq) {
  return(sapply(nseq, factorial))
}

MyFactorial2 <- function(nseq) {
  return(sapply(nseq,
    function(x) Reduce('*', seq(x))))
}

microbenchmark(
  factorial(c(5, 8, 10)),
  MyFactorial1(c(5, 8, 10)),
  MyFactorial2(c(5, 8, 10))
)
```

- In fact, `sapply` and `Reduce` do not really help with efficiency.
- May also use `Sys.time()` or `system.time()` to time R.

2.7.2 Python

```
import functools, operator, math
print(math.factorial(5))
```

```
def MyFactorial1(nseq):
  return [math.factorial(x) for x in nseq]
```

```
def MyFactorial2(nseq):
  return [functools.reduce(operator.mul,
    range(1, x+1)) for x in nseq]
```

```
print(MyFactorial1([5, 10]))
print(MyFactorial2([5, 10]))
```

```
import timeit
mysetup = '''
import functools, operator, math
def MyFactorial1(nseq):
  return [math.factorial(x) for x in nseq]
def MyFactorial2(nseq):
  return [functools.reduce(operator.mul,
```

```
range(1, x+1)) for x in nseq]
'''
```

```
print(timeit.timeit(setup=mysetup,
    stmt='MyFactorial1([5, 10])', number=10000))
```

```
print(timeit.timeit(setup=mysetup,
    stmt='MyFactorial2([5, 10])', number=10000))
```

3 Optimizing your code

- Vectorization.
 - Simple example: `sum` is faster than a `for` loop.
 - Matrix algebra is an important example of vectorization.
 - Don't forget some functions like `diff()`, `cumsum()`.
 - `rowSums()` is faster than `apply(x, 1, sum)`.
 - `crossprod(x, y)` is typically faster than `t(x) %*% y`.
 - In Python the same idea is NumPy: `A.sum(axis=1)`, `np.diff`, `np.cumsum`, `A.T @ B`. A Python `for` loop over an array is the slowest thing on this page.
- Try to find the computation bottleneck (either by algorithm or by profiling (a later tech-guide topic)).
- Avoid repeated computation:
e.g., `t(a - b) %*% A %*% (a - b)`.
- Avoid copies.
 - Use those functions on a minimum level: `c`, `append`, `rbind`, `cbind`. Create an empty vector (matrix) and then write a loop.
 - Python has the reverse problem. A NumPy slice does not copy, so writing into the slice writes into the original array. The R habit of assuming a copy gives wrong answers here, not slow ones. See Section 2.2.
 - `np.shares_memory(A, col)` tells you whether two arrays read the same block of memory. Write `.copy()` when you mean to modify.
- High-performance language (a later Tech Guide topic).
- Parallel computing (a later Tech Guide topic).

4 Coding style

4.1 Coding standards — good coding style

4.1.1 R

- The advanced R. <http://adv-r.had.co.nz/Style.html>
- The tidyverse style guide. <https://style.tidyverse.org/>
- R package: **styler**, including an RStudio add-in.
- R package **FormatR**: <https://yihui.org/formatr/#2-reformat-r-code>.

4.1.2 Python

- PEP 8, the official style guide. <https://peps.python.org/pep-0008/>
- Google’s Python style guide. <https://google.github.io/styleguide/pyguide.html>
- **black** reformats a file with no options to argue about. <https://black.readthedocs.io/>
- **ruff** checks style and common mistakes, and is fast. <https://docs.astral.sh/ruff/>

Both communities have converged on the same advice: pick one formatter, run it on save, and stop discussing spaces.

5 Plot

5.1 The state data

- This data sets related to the 50 states of the United States of America.
- Variables names:
 - **state.abb**: character vector of 2-letter abbreviations for the state names.
 - **state.area**: numeric vector of state areas (in square miles).
 - **state.center**: list with components named ‘x’ and ‘y’ giving the approximate geographic center of each state in negative longitude and latitude. Alaska and Hawaii are placed just off the West Coast.
 - **state.division**: factor giving state divisions (New England, Middle Atlantic, South Atlantic, East South Central, West South Central, East North Central, West North Central, Mountain, and Pacific).
 - **state.name**: character vector giving the full state names.
 - **state.region**: factor giving the region (Northeast, South, North Central, West) that each state belongs to.
 - **state.x77**: matrix with 50 rows and 8 columns giving the following statistics in the respective columns.

```
help(state, package="datasets")
```

The data ships with R. For the Python tab it was written out once to `data/state.csv`, with the five columns the slides use.

5.2 Building a data frame

5.2.1 R

```
data(state)
statedf <- data.frame(abb=state.abb,
  div=state.division,
  reg=state.region,
  state.x77[,c("Population", "Area")])
```

```
head(statedf[, c("Population", "Area")])
```

5.2.2 Python

```
import pandas as pd
statedf = pd.read_csv("data/state.csv")
```

```
statedf[["Population", "Area"]].head()
```

5.3 Functions operating on factors

5.3.1 R

```
is.factor(statedf[, "div"])
levels(statedf[, "div"])
```

5.3.2 Python

A pandas column of strings is just text. To get a factor you build a `Categorical`, and you have to give the level order yourself; pandas will not know that “New England” comes first.

```
divs = ["New England", "Middle Atlantic", "South Atlantic",
        "East South Central", "West South Central", "East North Central",
        "West North Central", "Mountain", "Pacific"]
regs = ["Northeast", "South", "North Central", "West"]
statedf["div"] = pd.Categorical(statedf["div"], categories=divs)
statedf["reg"] = pd.Categorical(statedf["reg"], categories=regs)

print(statedf["div"].dtype.name)
print(list(statedf["div"].cat.categories))
```

5.4 Group summaries by a factor

`tapply` was in the list of apply functions but never used. This is its job: one number per level of a factor.

```
tapply(statedf[, "Population"], statedf[, "reg"], mean)
```

🐞 Vibe — the pandas version is not on this page. Ask a model for it, run it, and compare with the four numbers above. Starter file: [vibe/s2p3_tapply.py](#). Done when the two tables agree to four decimals, in the same order.

The order is the part worth watching. R prints the regions in factor level order. pandas will sort them alphabetically unless the column is a `Categorical`, so a translation that looks right can still put the rows in the wrong places.

5.5 Using factors for plotting and model fitting

5.5.1 R

```
boxplot(Population ~ div, data=statedf)
boxplot(Population ~ div, data=statedf,
        pars=list(cex.axis=0.75))
```

5.5.2 Python

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
statedf.boxplot(column="Population", by="div", ax=ax, rot=90)
plt.show()
```

The tilde in R is a formula: it says “split **Population** by **div**”. Python has no formula notation in this position, so you name the column and the grouping column separately.

The R function `dev.copy2eps` copies a plot from the screen to a postscript file.

We may also open a graphic device (say a postscript file), produce the plot, and then turn off the graphic device, saving the plot to the postscript file, for instance. A browser has no file system to write to, so run these two on your own machine.

5.5.3 R

```
dev.copy2eps(file="boxplotstatepop.eps", horizontal=T)

setEPS()
postscript(file="boxplotstate", height=4, width=6)
boxplot(Population ~ div, data=statedf)
dev.off()
```

5.5.4 Python

```
fig, ax = plt.subplots(figsize=(6, 4))
statedf.boxplot(column="Population", by="div", ax=ax, rot=90)
fig.savefig("boxplotstate.eps")
```

R thinks of a device you open, draw on, and close. Matplotlib thinks of a figure object you build and then save; the file type comes from the extension.

5.6 Plotting functions

5.6.1 R

- **High-level plotting functions** create a new plot on the graphics device, possibly with axes, labels, titles and so on.
- **Low-level plotting functions** add more information to an existing plot, such as extra points, lines and labels.
- **Interactive graphics functions** allow you interactively add information to, or extract information from, an existing plot, using a pointing device such as a mouse.

5.6.2 Python

- Matplotlib splits the same work in two: `plt.subplots()` creates the **figure and axes**, then every method of the axes object adds one more thing to it.
- There are no separate high-level and low-level functions. `ax.scatter`, `ax.plot`, `ax.text`, and `ax.legend` are all methods of the same object, and you call them in any order.
- `plt.show()` at the end draws the figure. On this page it is what makes the plot appear under the block.

5.7 Example of high-level function: Plot

`plot` is a generic plotting function whose behavior is determined by the class of the object(s) to which it is applied.

- Argument is factor: bar graph of counts of each level.

5.7.1 R

```
plot(statedf[, "div"], cex.axis=0.75,  
     main="Number of States per Division")
```

5.7.2 Python

```
fig, ax = plt.subplots()  
statedf["div"].value_counts().reindex(divs).plot.bar(ax=ax)  
ax.set_title("Number of States per Division")  
plt.show()
```

- Arguments are two numeric vectors: scatterplot with first vector on x-axis.

5.7.3 R

```
plot( statedf[, "Area"], statedf[, "Population"],  
      xlab="Area in Square Miles",  
      ylab="Population in thousands")
```

5.7.4 Python

```
fig, ax = plt.subplots()
ax.scatter(statedf["Area"], statedf["Population"])
ax.set_xlabel("Area in Square Miles")
ax.set_ylabel("Population in thousands")
plt.show()
```

- Argument is a data frame: scatterplot matrix.

5.7.5 R

```
plot(statedf)
```

5.7.6 Python

```
pd.plotting.scatter_matrix(statedf, figsize=(7, 7))
plt.show()
```

R draws all five columns, factors included. `scatter_matrix` keeps only the numeric ones, so the Python panel is two by two.

- Plotting one object against each object in an expression.
 - Object to left of “~” will be on y-axis.

5.7.7 R

```
par(mfrow=c(1,2))
plot(Population ~ Area + reg, data=statedf)
```

5.7.8 Python

```
fig, axes = plt.subplots(1, 2, figsize=(9, 4))
axes[0].scatter(statedf["Area"], statedf["Population"])
statedf.boxplot(column="Population", by="reg", ax=axes[1], rot=45)
plt.show()
```

One R call produced both panels, because `plot` looked at the class of each right-hand variable and chose a scatterplot for the numeric one and a boxplot for the factor. In Python you make that choice yourself, one panel at a time.

5.8 Log scales

5.8.1 R

```
plot(state.x77[, "Area"],  
     state.x77[, "Population"], log="x")
```

```
plot(state.x77[, "Area"],  
     state.x77[, "Population"], log="xy")
```

5.8.2 Python

```
fig, ax = plt.subplots()  
ax.scatter(statedf["Area"], statedf["Population"])  
ax.set_xscale("log")  
plt.show()
```

```
fig, ax = plt.subplots()  
ax.scatter(statedf["Area"], statedf["Population"])  
ax.set_xscale("log"); ax.set_yscale("log")  
plt.show()
```

5.9 Low-level plotting functions

Add extra information (such as points, lines or text) to the current plot.

- `points(x,y)`.
- `lines(x,y)`.
- `text(x,y,labels,...)`.

5.9.1 R

```
attach(statedf)
plot(Area, Population, type="n")
text(Area, Population, abb)
detach(statedf)
```

5.9.2 Python

```
fig, ax = plt.subplots()
ax.scatter(statedf["Area"], statedf["Population"], alpha=0)
for a, p, ab in zip(statedf["Area"], statedf["Population"],
                    statedf["abb"]):
    ax.text(a, p, ab, ha="center", va="center", fontsize=7)
plt.show()
```

- `legend(x, y, legend, ...)`.

We introduce `attach()` here for your information, but we should try to avoid using `attach()` in practice. Python has no counterpart, which is one problem fewer.

5.9.3 R

```
statedf = statedf[order(statedf[, "Area"]), ]
attach(statedf)
plot(Area, Population)
lines(lowess(Population ~ Area))
lines(lowess(Population ~ Area, f=0.25), lty=2)
legend(400000, 15000,
      legend=c("f=2/3", "f=1/14"), lty=1:2)
detach(statedf)
```

5.9.4 Python

```
import statsmodels.api as sm
d = statedf.sort_values("Area")
fig, ax = plt.subplots()
ax.scatter(d["Area"], d["Population"])
ax.plot(*sm.nonparametric.lowess(d["Population"], d["Area"],
                                frac=2/3).T, label="frac=2/3")
```

```
ax.plot(*sm.nonparametric.lowess(d["Population"], d["Area"],
    frac=0.25).T, ls="--", label="frac=1/14")
ax.legend()
plt.show()
```

Both `lowess` functions come from the same 1979 Cleveland algorithm, so the two curves should agree. The smoothing fraction is called `f` in R and `frac` in `statsmodels`.

5.10 Interactive graphic functions

- `locator(n, type)`.
 - Waits for the user to select `n` locations on the current plot using the left mouse button.
 - Returns the locations of the points selected as a list with two components `x` and `y`.

```
text(locator(2), "Outlier")
```

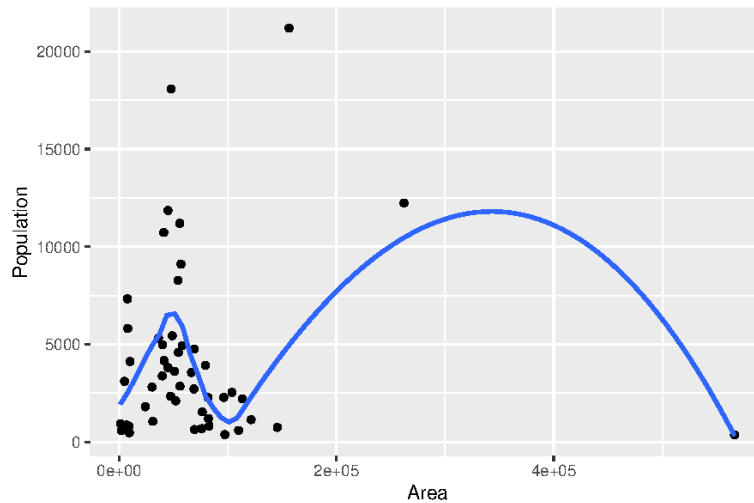
The Python counterpart is `plt.ginput(2)`. Neither one runs here: both need a mouse on a real graphics window, which a web page does not give them.

5.11 ggplot2 package

5.11.1 R

Run in Codespace — `ggplot2` is not installed in the browser session.

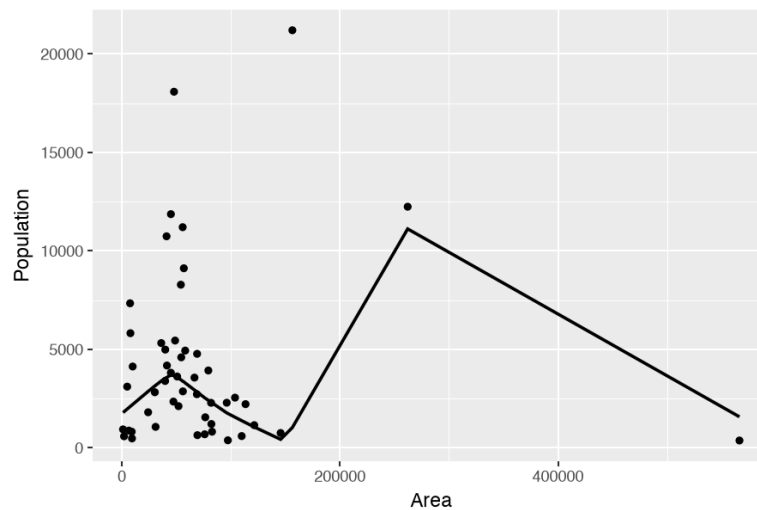
```
library(ggplot2)
p <- ggplot(statedf, aes(Area, Population)) +
  geom_point()
p + geom_smooth(method="loess", se=FALSE)
```



5.11.2 Python

Run in Codespace — `plotnine` is not installed in the browser session either.

```
from plotnine import ggplot, aes, geom_point, geom_smooth
p = ggplot(statedf, aes("Area", "Population")) + geom_point()
p + geom_smooth(method="lowess", se=False)
```



`plotnine` is a port of the grammar of graphics to Python: the same `ggplot`, `aes`, and `geom_` names, with the column names written as strings.